

Day 3 - DSA Array Sliding Window

ITSRUNTYM

1. Definition of Sliding Window

The Sliding Window is a technique used to solve problems that involve contiguous segments or subarrays of a given array or string.

- It involves maintaining a window (a range or segment) defined by two pointers (usually start and end) that slide over the input data.
- The window can either be fixed size or variable size.
- Instead of recalculating everything for every window, it efficiently updates results as the window moves forward by adding the new element and removing the old element.

Core Idea:

Instead of computing the result again and again for every window (subarray or substring), reuse the previous computation by sliding a "window" (range of elements) forward.

2. Where Sliding Window Applies

Use it when:

- You need to work with **contiguous elements** in an array or string.
- You're asked for:
 - Max/min/average **sum** of subarrays.
 - Longest/shortest **substring** with some properties.
 - Count/frequency of elements **in a window**.
- Your **brute-force solution is $O(n^2)$** , and you want to optimize it.

4. Types of Sliding Window

A. Fixed Size Window (Window of size K)

Example Problem:

"Find the maximum sum of a subarray of size k."

Approach:

1. Initialize the sum of the first window of size k.
2. Slide the window one step at a time:
 - **Subtract** the element that goes out of the window.
 - **Add** the new element that comes into the window.

Array: [1, 3, 2, 5, 6, 1, 4]

Window: [2, 5, 6] → sum = 13

↑ ↑
start end

Code:

```
public int maxSumSubarray(int[] arr, int k) {
    int left = 0;
    int windowSum = 0;
    int maxSum = 0;

    for (int right = 0; right < arr.length; right++) {

        // Shrink the window before adding arr[right]
        if (right - left == k) {
            windowSum -= arr[left];
            left++;
        }

        // Now add arr[right] to windowSum
        windowSum += arr[right];

        // If window size is exactly k, update maxSum
        if (right - left + 1 == k) {
            maxSum = Math.max(maxSum, windowSum);
        }
    }
}
```

```

    }

    return maxSum;
}

```

Time: $O(n)$ | Space: $O(1)$

B. Variable Size Window

Example Problem:

"Find the length of the smallest subarray with sum $\geq$ target."

Approach:

1. Use two pointers start and end.
2. Expand the window by moving end to increase the sum.
3. When sum $\geq$ target:
 - Shrink the window by moving start to reduce the size **while maintaining the condition**.
4. Track the **minimum length** window.

Array: [2, 3, 1, 2, 4, 3], **target = 7**

Start-End Movement:

→ sum = 2 + 3 + 1 + 2 = 8 $\geq$ 7 → shrink from left → size = 4

↑ ↑
 start end

Code:

```

public int minSubArrayLen(int target, int[] nums) {
    int start = 0, sum = 0, minLen = Integer.MAX_VALUE;
    for (int end = 0; end < nums.length; end++) {
        sum += nums[end];
        while (sum >= target) {
            minLen = Math.min(minLen, end - start + 1);
            sum -= nums[start];
            start++;
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}

```

Time: $O(n)$ | Space: $O(1)$

5. How to Identify Sliding Window Problems

Look for:

- Keywords like:
 - "subarray", "substring", "window", "contiguous", "range".
- Constraints like $n \leq 10^5 \rightarrow$ Brute-force is too slow.
- Asked for:
 - Max/min/average sum.
 - Longest/shortest substring with condition.
 - Number of distinct or frequent elements in a window.
- Naive approach involves nested loops checking all subarrays.

6. Benefits of Sliding Window

Feature	Benefit
Efficiency	Reduces time from $O(n^2)$ to $O(n)$
Simple Logic	Easy to understand and write once pattern is grasped
Low Memory	Often uses constant or linear extra space
Versatile	Can be used for sums, averages, frequency, and other range properties
Real-time	Ideal for stream processing and online algorithms

Summary

The Sliding Window pattern is a must-know for coding interviews and DSA problems involving ranges of contiguous elements. It lets you solve problems more efficiently by:

- Maintaining a running result as the window slides.

- Avoiding recomputation.
- Optimizing brute-force solutions.

No.	Problem Name	Difficulty	Platform	Link
1	Maximum Average Subarray I	Easy	LeetCode	Link
2	Minimum Size Subarray Sum	Medium	LeetCode	Link
3	Longest Substring Without Repeating Characters	Medium	LeetCode	Link
4	Permutation in String	Medium	LeetCode	Link
5	Find All Anagrams in a String	Medium	LeetCode	Link
6	Sliding Window Maximum	Hard	LeetCode	Link
7	Longest Repeating Character Replacement	Medium	LeetCode	Link
8	Count Number of Nice Subarrays	Medium	LeetCode	Link

9	Longest Substring with At Most K Distinct Characters	Medium	LeetCode	Link
10	Substring with Concatenation of All Words	Hard	LeetCode	Link
11	Maximum Number of Vowels in a Substring of Given Length	Medium	LeetCode	Link
12	Count Good Substrings	Easy	LeetCode	Link
13	Longest Substring Without 3 Identical Consecutive Characters	Medium	LeetCode	Link
14	Maximum Erasure Value	Medium	LeetCode	Link
15	Max Consecutive Ones III	Medium	LeetCode	Link